

树上统计

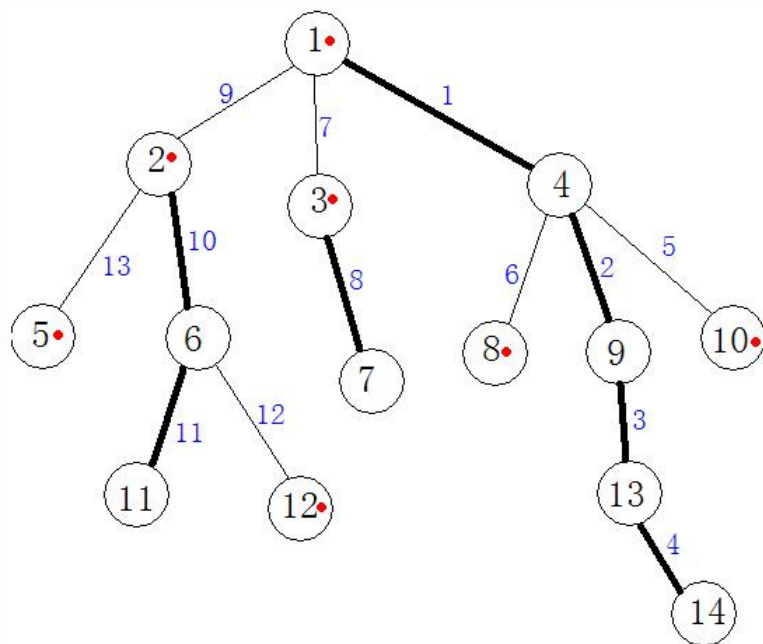
在前面我们学习了两种树上简单统计的办法：

第一种是基于不带修改的，我们可以直接用 lca 来解决。

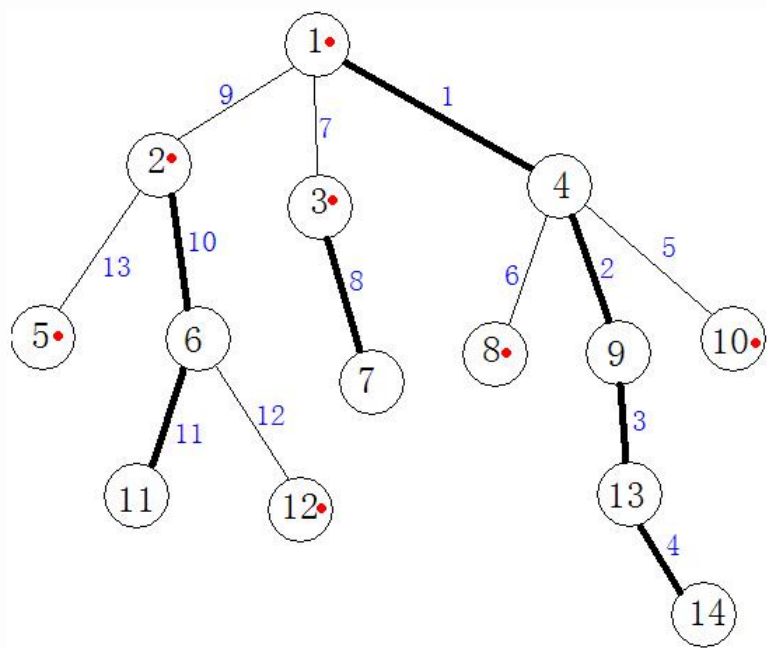
第二种是基于两点之间的最短路径修改的，我们可以用树上差分来解决。但是树上差分也有一个问题。第一，他不能不停的询问，只能是离线的，如果在线，每次查询的时间复杂度为 $O(n)$ ，第二，他不能处理任意的修改，所以树上差分他有很大的局限性。所以我们现在要学习一种高级的数据结构来维护树上的任意修改——树链剖分。

树链剖分

树链剖分的思想就是把一棵树拆分成很多连续的链，再把这些链合成一棵线段树，这样我们的修改和查询就可以在很快的时间内完成了。当然，每条链越长，链的总量越少，那么效率肯定就越高。



树链剖分

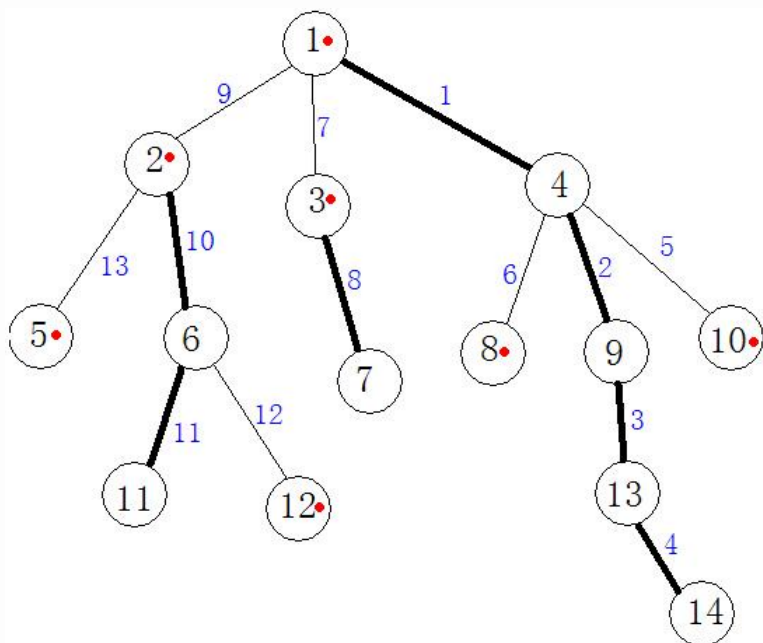


比如该图中，如果粗线表示连续的一条链，细线表示一条单链，那么如果我要查询 $11 \rightarrow 14$ 的路径就可以转换成查询

$11 \rightarrow 2, 2 \rightarrow 1, 1 \rightarrow 14$ 这三段区间在线段树上查询了。这样效率就提高很多了。

树链剖分

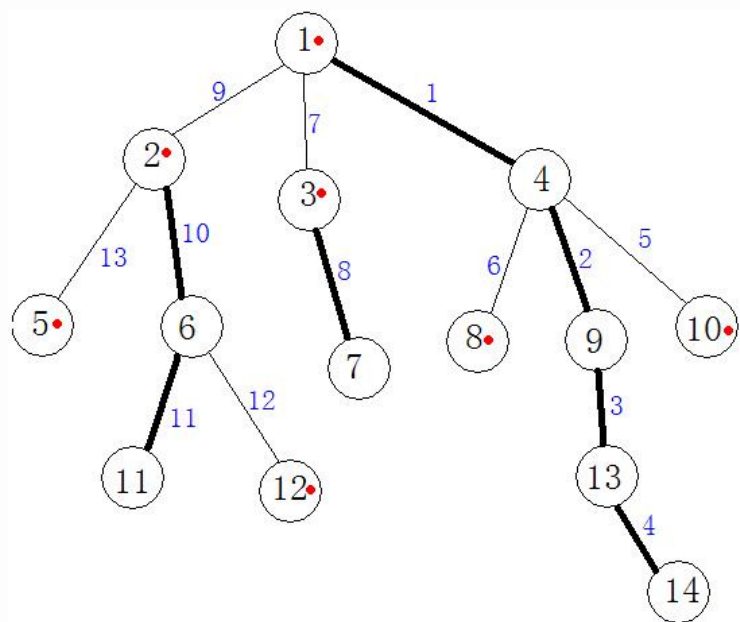
树链剖分的方式有很多种，目前常用的效率也比较高的一种是轻重链剖分。即对于每一个结点，选择结点数最多的儿子为他的重儿子，根结点和重儿子的边称为重边，其他儿子都为他的轻儿子，根结点和轻儿子的边称为轻边。



树链剖分

为什么要这么划分呢？

因为我们观察图就会发现，重儿子所在的路径是子树中路径最长的，如果我们让这一条重链上点的编号连续，那么一条重链实际上就是线段树上一段连续的区间。这样我们把一个树拆分成多条链的时候得到的链就会最少，这样查询一条路径的时候分割成的区间就越少。



树链剖分

树链剖分的时间复杂度是多少呢？也就是说从任意一个结点到根结点最多经历的链有多少条？

我们来考虑一种最坏的情况，即从当前结点到根结点所经过的路径都是轻链，那么最多经过多少条轻链呢？

对于一个根结点，轻链结点的size < 重链结点的size，即 < 根结点size的一半，所以没经过一条轻链，那么节点数减半，所以如果经过了k条轻链到叶子结点，那么说明这棵树至少有 2^k 个结点，所以一棵节点数为n的树，按照轻重链剖分的方式最多 $\log n$ 次就可以找到根结点，所以最坏时间复杂度为 $\log n$

轻重链划分

所以树链剖分第一步就是轻重链划分：

对于重链来说：一条重链上的点的编号必须是连续的，这样我们才能在线段树上当做一整个区间来处理。

对于轻链来说：轻链是独立的，只有一条边，因为要照顾到重链连续，所以轻链就没有办法连续。

对于一个根结点 u 和其子节点 v

定义 u 点的重儿子为 $\text{son}[u]$ ：

```
if(size[v]>size[son[u]])
```

```
son[u]=v;
```

轻重链划分

因为我们需要知道每个结点的子结点的个数，以及每个结点的重儿子的编号，所以我们需要先求出来：

```
void dfs1(int u,int f)
{
    fa[u]=f; size[u]=1;
    for(int i=head[u];i;i=edge[i].next)
    {
        int v=edge[i].to;
        if(v!=f)
        {
            deep[v]=deep[u]+1;
            dfs1(v,u);
            size[u]+=size[v];
            if(son[u]==0 || size[v]>size[son[u]])
                son[u]=v;
        }
    }
}
```


轻重链划分

接下来我们重新对每个点进行编号，让每条重链上的点的编号连续，每棵子树上的点编号也连续(让树剖支持整棵子树修改)，同时，我们还需要知道每条重链的顶点，因为如果一个点就在这条重链上，那么我们可以直接跳到重链的顶点去。所以这里我们需要开几个数组：

top[]: 记录每一个点所在链的顶点，轻链的顶点是他本身。

pos[x]; 以前编号为x的点现在的编号

tid[x]; 现在编号为x的点以前的编号

轻重链划分

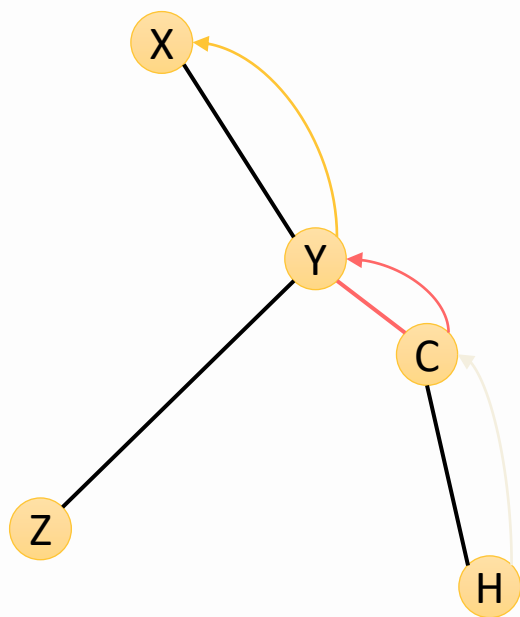
```
void dfs2(int u,int tp)
{
    top[u]=tp;
    num++;
    pos[u]=num;//原来的点x新赋予的编号为num
    tid[num]=u;//现在编号为num的点原来的编号
    if(son[u]==0)//如果是叶子结点
        return ;
    dfs2(son[u],tp);//tp表示当前重链的顶点。
    for(int i=head[u];i;i=edge[i].next)
    {
        int v=edge[i].to;
        if(v!=son[u]&&v!=fa[u])
        {
            dfs2(v,v);//轻链的起点就是它本身
        }
    }
}
```

建树

接下来就很简单了啊，建一颗sum线段树，建法和以前一摸一样：但是要注意线段树中的区间标号代表的是线段的编号，所以对于每一个位置 x 的值，实际上是 $tid[x]$ 的值。

分割

假设题目中要求我们修改 $u \rightarrow v$ 的路径，但是很大的可能性， $u \rightarrow v$ 没有在一条重链上，而是由多条重链和轻链组成的。所以我们需要先分割出来再去用线段树去查询。



比如我要修改 $Z \rightarrow H$ ，但是这两个点又不是一条重链上，我们可以参照lca的办法，都往上跳，准确的说，是深度大的往深度小的跳，如果是重儿子直接跳到重链的顶端，如果是轻儿子跳到父亲节点。最终一定会相遇。

分割

```
void trim(int x,int y,long long z)
{
    while(top[x]!=top[y])//如果x,y不在一条重链上
    {
        if(deep[top[x]]<deep[top[y]])
        {
            int tmp=x;x=y;y=tmp;
        }
        update(1,pos[top[x]],pos[x],z);//查询跳的这一段sum
        x=fa[top[x]];//说明上面是轻链了，找父亲节点
    }
    if(deep[x]>deep[y])//说明此时在一条重链上
    {
        int tmp=x;x=y;y=tmp;
    }
    update(1,pos[x],pos[y],z);
}
```

查询

如果要查询区间 $[l, r]$ ，其实和前面的修改是一样的，都是先找到当前区间 $[l, r]$ 在线段树中是由那几段区间构成的。只是把线段树的修改函数改成线段树的查询函数就可以了。

子树修改

树链剖分是支持子树修改的，也就是修改以 u 为根的整棵子树，比如子树的值都 $+x$ 。

我们在前面定义编号的时候，重链的编号是连续的，因为我们是dfs，所以实际上每个子树的编号总的来说也是连续的，所以实际上 u 的子树在线段树上的区间就是 $[\text{pos}[u], \text{pos}[u] + \text{size}[u] - 1]$



练习题

洛谷	3384	轻重链剖分
洛谷	3038	Grass Planting G
洛谷	2486	染色
洛谷	2146	软件包管理器
洛谷	1505	旅游
洛谷	4315	月下毛景树

